

Version: 1.1

Audience: Security engineers and .NET architects

Date: May 2026

Status: Demo / educational

Executive Summary

This project demonstrates a secure-by-default shape for a small MCP-style release-assessment server and a rule-based console client. It is built with .NET 8 and focuses on auth boundaries, scope checks, consistent tool discovery, auditable tool calls, and negative-path tests.

The system is not production-ready identity infrastructure. The token issuer and secrets are intentionally local demo mechanisms.

Components

Component	Responsibility	Security control
---	---	---
'SecureMcpServer'	ASP.NET Core Minimal API host for auth, SSE, JSON-RPC messages, and tool discovery	JWT bearer auth and scope policies
'DemoTokenService'	Issues and validates demo JWTs	issuer, audience, signing key, lifetime validation
'ReleaseDataService'	Serves release status, dependencies, vulnerability data, and release approval	checks 'mcp:tools:release' before approval
'SecurityAuditLogger'	Logs auth success/failure, tool calls, and tool errors	structured application logs
'SecureMcpClient'	Console client with a simple rule-based planner	obtains token and delegates auth decisions to the server
'demo-data.json'	Self-contained release data for deterministic responses	copied to build output

Endpoint Model

Endpoint	Method	Purpose	Auth
---	---	---	---
'/auth/token'	POST	Demo-only JWT issuance	validates requested scopes and demo release secret
'/mcp/sse'	GET	SSE endpoint advertisement	requires 'mcp:tools'
'/mcp/messages'	POST	JSON-RPC 'tools/list' and 'tools/call'	requires 'mcp:tools'; 'approve_release' also requires 'mcp:tools:release'
'/mcp/tools'	GET	Declarative tool discovery	requires 'mcp:tools'

The SSE endpoint advertises '/mcp/messages' without embedding the bearer token in the URL. Clients should continue to send the token in the 'Authorization' header.

Tool Contract

'/mcp/tools' and 'tools/list' expose the same four implemented tools:

Tool	Purpose	Required arguments	Additional scope
---	---	---	---
'get_release_status'	Returns release readiness and approvers	'component', 'version'	none
'get_dependencies'	Returns component dependencies	'component', 'version'	none
'check_security_vulnerabilities'	Returns demo vulnerability findings	'component', 'version'	none
'approve_release'	Approves a release and returns an approval ticket	'component', 'version'	'mcp:tools:release'

Auth Flow

```

'''text
Client
|
| POST /auth/token { clientId, scopes, clientSecret? }
v
SecureMcpServer
|
| validate requested scopes
| require demo-release-secret for mcp:tools:release
v
JWT access token

Client
|
| Authorization: Bearer <token>
| POST /mcp/messages tools/call
v
SecureMcpServer
|
| validate JWT
| require mcp:tools
| require mcp:tools:release for approve_release
| audit call
v
JSON-RPC result or JSON-RPC error
'''

```

Negative paths:

- missing or invalid token returns 401
- valid token without 'mcp:tools' returns 403
- 'approve_release' without 'mcp:tools:release' returns 403
- privileged token issuance without the demo release secret returns 403
- unknown scopes return 400

Data Flow

'ReleaseDataService' loads 'demo-data.json' from the server output directory. If the file is not present, it falls back to an embedded default payload so the demo remains runnable during local experiments.

Current demo releases:

- 'AnalyzerService' '2.4.1': ready, no vulnerabilities
- 'PaymentGateway' '5.8.0': blocked, includes 'CVE-DEMO-2026-0001'

Testing Strategy

The server tests use 'WebApplicationFactory<Program>' to exercise the actual ASP.NET Core pipeline. Coverage includes token issuance constraints, release-scope authorization, tool discovery consistency, and demo data responses.

The client tests cover the rule-based planner's known and fallback parsing paths.

Run:

```
'''bash
dotnet test --configuration Release
'''
```

Production Hardening

For real deployment:

1. Replace '/auth/token' with OAuth2/OIDC through a trusted provider.
2. Replace symmetric demo signing with RSA or ECDSA and key rotation.
3. Store secrets in a managed secret store.
4. Add rate limiting, request-size limits, and stricter JSON-RPC validation.
5. Store audit logs in a tamper-resistant sink.
6. Add end-to-end tests for the intended MCP transport and client behavior.

This architecture document is for educational and demonstration purposes only.